Introduction Couche Transport

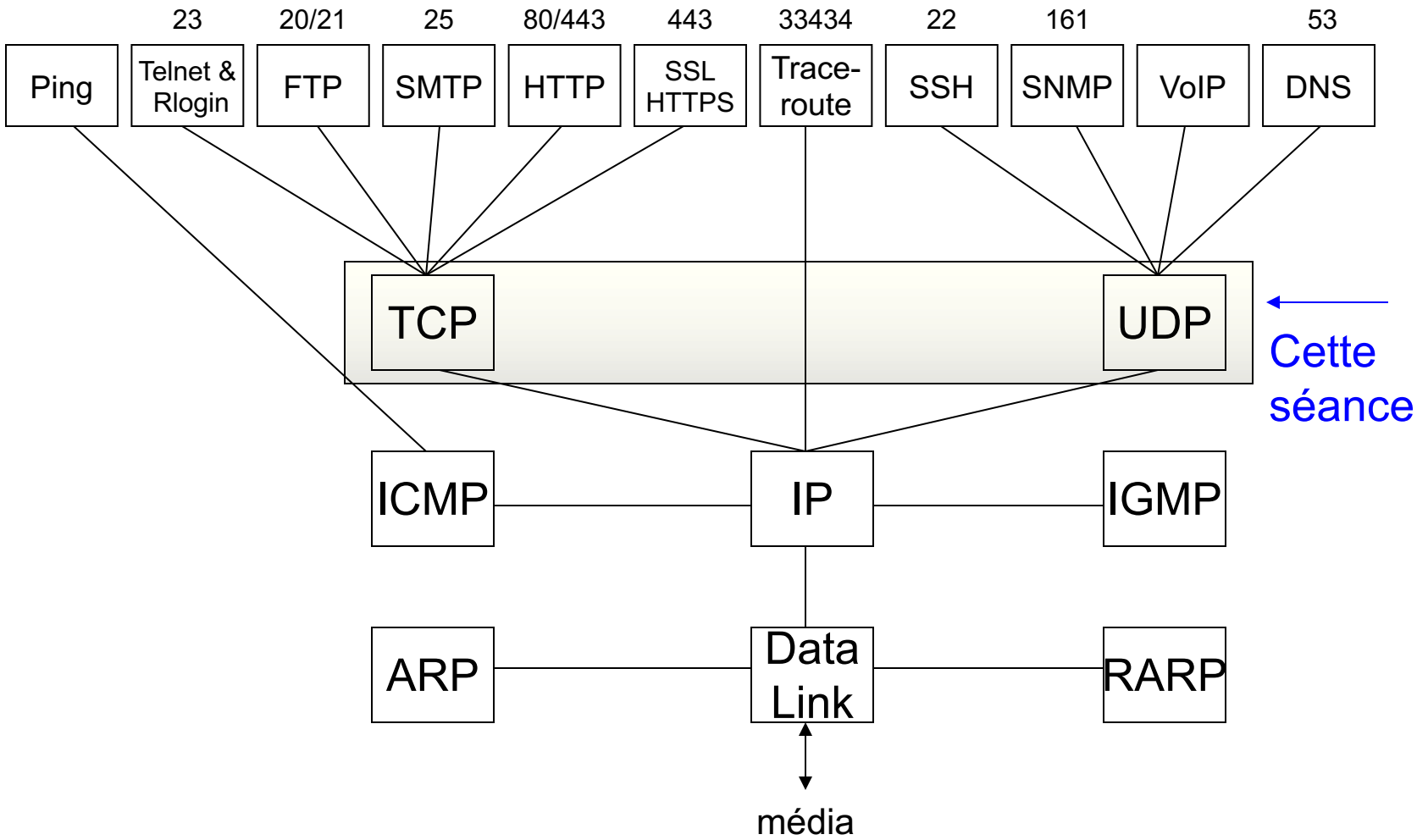
GEF330 – Hiver 2021
CRI-10

Mariam El Mezouar
Collège Militaire Royal du Canada
Génie électrique et génie informatique
Mariam.el-mezouar@rmc-cmr.ca
elmezouar.segfaults.net

Références

- Tanenbaum, § 6.2, § 6.3, § 6.4
- Stevens, W.R., TCP/IP Illustrated Vol I, Addison-Wesley
- QUIC: https://tools.ietf.org/html/draft-ietf-quic-transport22#section-1
- The State of QUIC and HTTP/3 in 2020: https://www.fastly.com/blog/state-of-quic-and-http3-2020

Situation


23
20/21
25
80/443
443
33434
22
161
53
Ping
Telnet & Rlogin
FTP
SMTP
HTTP
SSL HTTPS
Trace-route
SSH
SNMP
VoIP
DNS
TCP
UDP
Cette séance
ICMP
IP
IGMP
ARP
Data Link
RARP
média

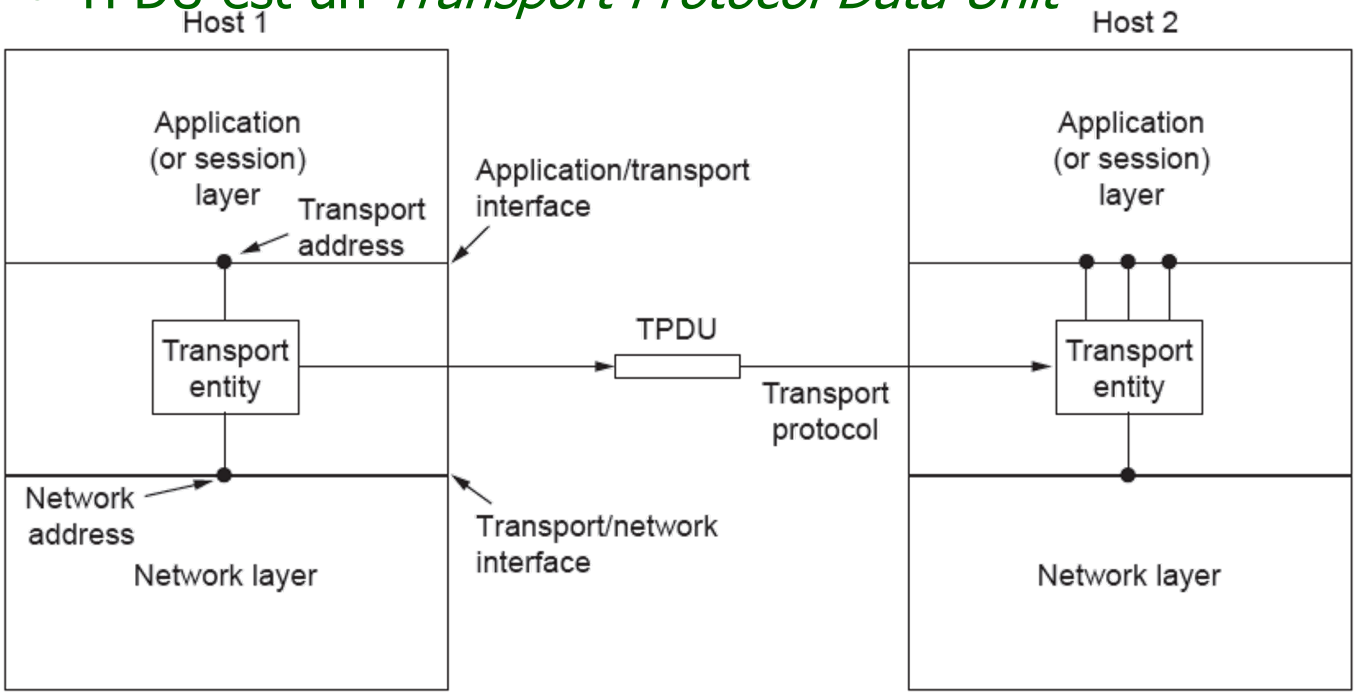
Aperçu

- Service de transport
- Éléments de protocole de transport
- Protocole Internet – UDP
  - Datagrammes
  - Format de l'en-tête
  - Applications
- Protocole Internet – TCP
  - Communications fiables
  - Format de l'en-tête
  - Connexion de session
  - Applications
- Faire évoluer Internet: QUIC

Services fournis aux couches supérieures (1)

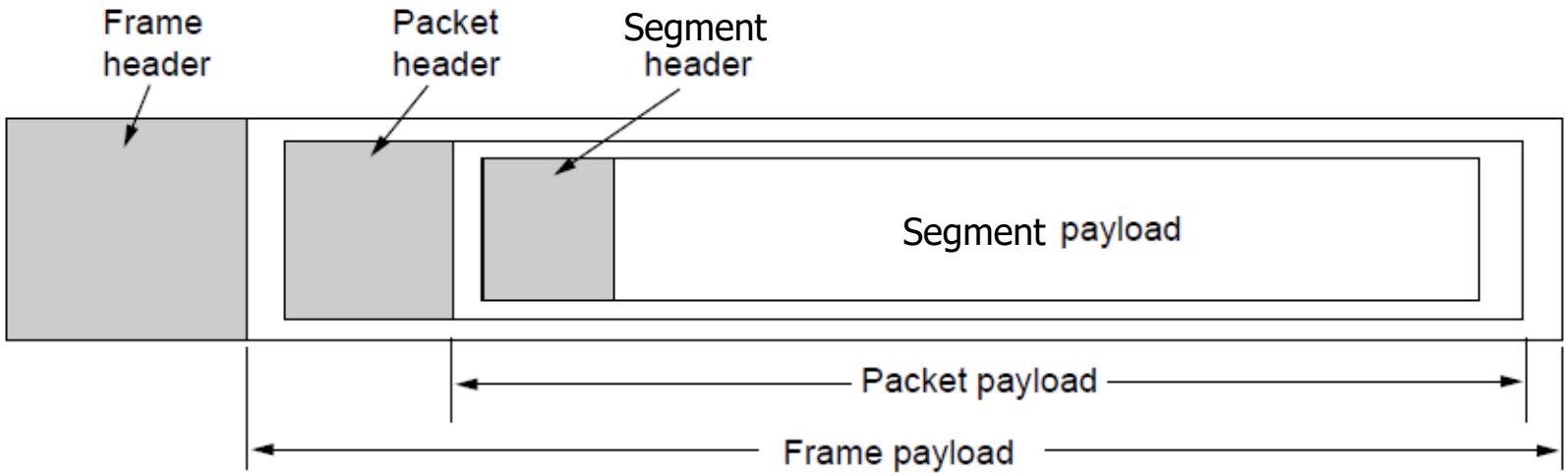
La couche transport ajoute la fiabilité à la couche réseau

- Offre service sans connexion (par ex. UDP) et avec connexion (par ex. TCP) aux applications
  - TPDU est un *Transport Protocol Data Unit*



Services fournis aux couches supérieures (2)

La couche transport envoie des *segments* dans les paquets (à l'intérieur de trames)



Éléments de protocoles de transport

- Adressage
- Établissement de connexion
- Relâchement de connexion
- Contrôle d’erreurs et de flots
- Multiplexage
- Sauvetage d’un plantage

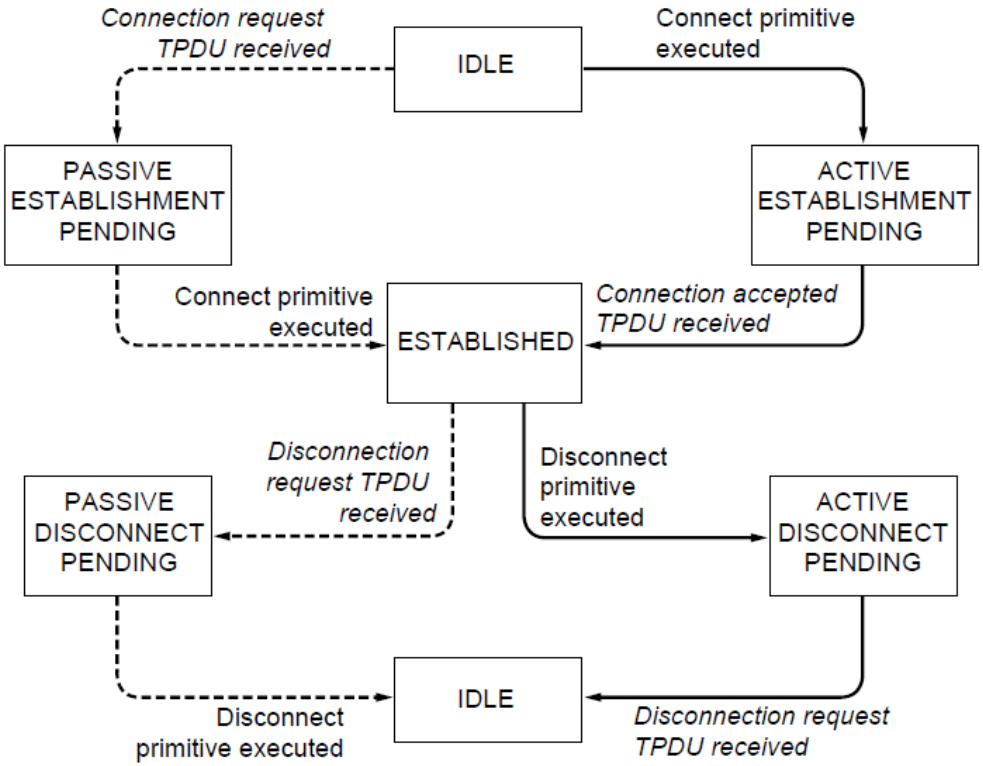
Primitives de service transport (1)

- Primitives que les applications peuvent appeler pour un service avec connexion simple:
  - Client appelle `CONNECT`, `SEND`, `RECEIVE`, `DISCONNECT`
  - Serveur appelle `LISTEN`, `RECEIVE`, `SEND`, `DISCONNECT`

Primitive	Segment sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

Primitives de service transport (2)

Diagramme d'état pour un service avec connexion simple



Les lignes solides (à droite) montrent la séquence d'états du client

Lignes pointillées (à gauche) montrent la séquence d'états du serveur

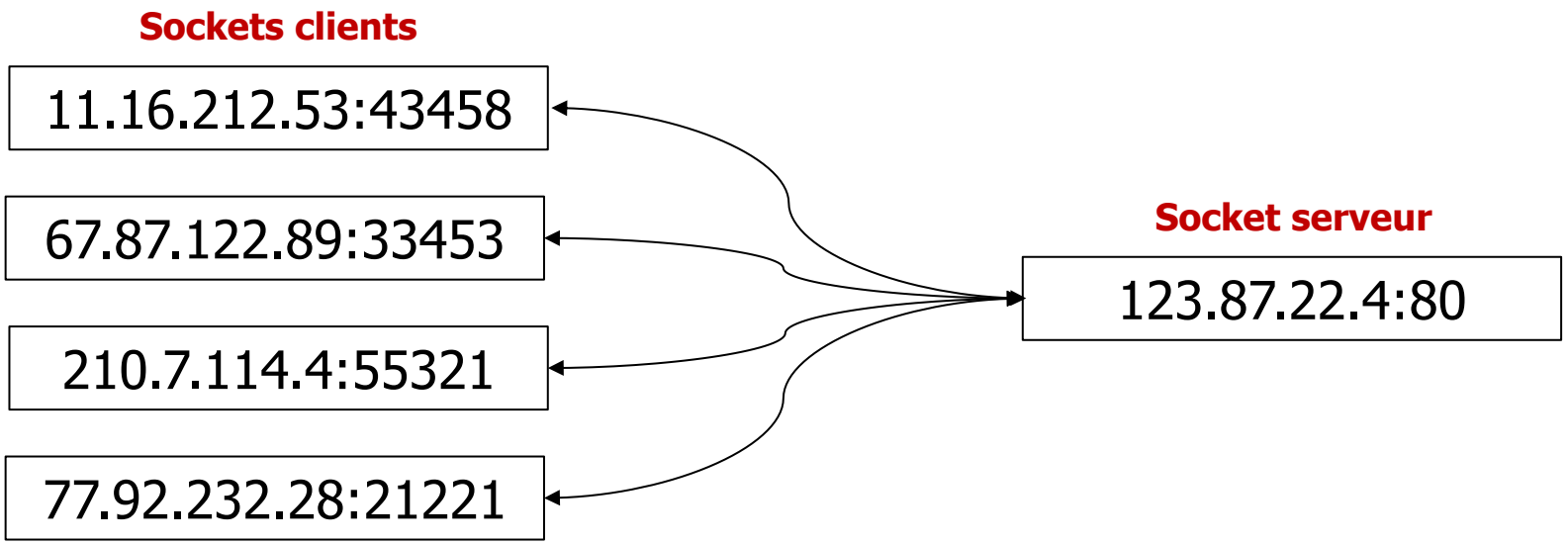
Les transitions (en italiques) sont causées par l'arrivée de segments

Ports UDP

- Les services réseaux sont généralement connus selon les ports *usuels*
  - Le terme *réservé* n'est pas exact selon nous; une application peut accepter les connexions à n'importe quel port.
  - Essentiellement ports 1 à 1023 (Mais certains services utilisent des ports usuels situés au delà)
- Ports UDP & TCP usuels (Exemples)
  - DNS au port UDP 53 /Domain Name Service
  - Tftp au port UDP 69 /Trivial File Transfer Protocol
  - ntp au port UDP 123 /Network Time Protocol
  - http au port TCP 80 /Web Server
  - ftp au port TCP 21 /File Transfer Protocol
  - ssh au port TCP 22 /Secure Shell
- Les clients ne se préoccupent habituellement pas de leur port local utilisé localement; on les appels ports *éphémères*
  - 1024 – 65535
  - Assigné aléatoirement lorsque nécessaire.

Sockets

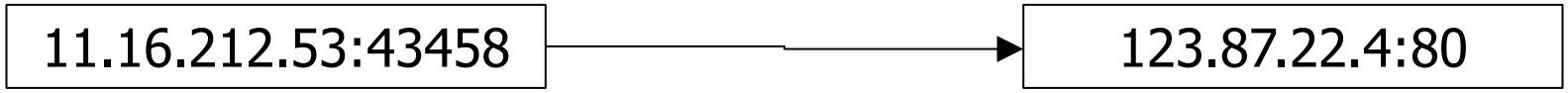
- La combinaison de l'adresse IP et du port TCP / UDP peut être appelée socket.
- Les sockets permettent de suivre facilement les différentes connexions réseau, même pour les serveurs avec des milliers de clients.



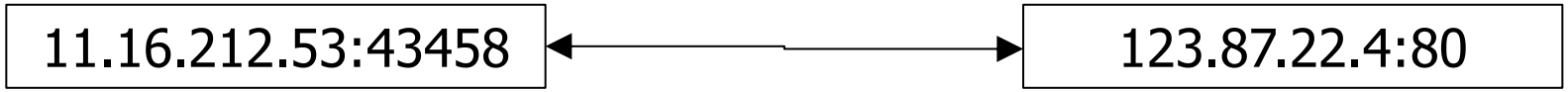
Flux & Conversations

- Le flux unidirectionnel de données du socket source au socket de destination peut être considéré comme un *flux*.
- Le flux bidirectionnel de données entre deux sockets peut être appelé une *conversation*.
  - Ce terme est principalement utilisé du point de vue de l'analyse du trafic réseau.

Flux 11.16.212.53:43458 > 123.87.22.4:80



Conversation 11.16.212.53:43458 <> 123.87.22.4:80



Un protocole sans connexions:

UDP

Datagrammes UDP

- UDP est un protocole de la couche transport simple, sans-connexions.
- Un datagramme UDP est encapsulé dans IP.

En-tête IP (20 octets)	En-tête UDP (8 octets)	Données UDP

- UDP est un protocole de couche transport peu fiable puisqu'il n'offre aucune garantie de la livraison de paquets;
  - Le protocole ne garantit pas la livraison de messages ICMP lorsqu'il y a une erreur
  - Les messages d'erreurs peuvent être abandonnés ou perdus; certaines applications ne génèrent pas de messages ICMP

Format En-tête UDP

bits	0-15	16-31
0	Source Port	Destination Port
32	UDP length	UDP checksum
64	Data	

- Le port destination définit le « service ».

Applications UDP

- Le protocole UDP est utile pour les situations où un protocole de transport simple suffit.
  - En autres mots dans des situations où un flux de donnée parfait n'est pas nécessaire et où la perte d'un paquet de temps à autre est acceptable lorsqu'on compare celle-ci à la surcharge du protocole TCP.
- Exemples:
  - Protocoles de routage RIP, OSPF
  - Les services de noms distribués de l'Internet: DNS.
  - Transport de voix en temps réel : RTP (VoIP)
  - Jeux en ligne

Un protocole avec connexions

TCP

TCP : Transmission Control Protocol

- Le TCP (Transmission Control Protocol) est un protocole complexe et à état (Statefull).
- TCP fournit un canal de communication à flux d'octets Full-Duplex, fiable et orienté connexion;
  - Orienté connexion c.-à-d. que les données d'application ne traversent pas le réseau tant qu'une connexion n'est pas établie,
  - Fiable c.-à-d. qu'il met en œuvre un schéma de demande de répétition automatique (ARQ) pour garantir l'absence de perte de données,
  - Flux d'octets c.-à-d. que les couches d'application échangent des données de manière transparente quelle que soit la taille des données, et
  - Full Duplex dans le sens que les données circulent de manière asynchrone dans les deux sens.

Communications fiables*

- La fiabilité de transmission de données au travers du réseau à la couche transport est atteinte de deux façons distinctes:
  1. Un mécanisme pour s'assurer que les données reçues sont les mêmes que les données transmises.
     - Accompli avec les sommes de contrôles (*checksum*) utilisées par UDP et TCP
  2. Un mécanisme pour s'assurer que chaque paquet transmis est délivré
     - Accompli par les systèmes de demande de répétition automatique (ARQ) utilisés par TCP

*Cette fiabilité présume qu'il n'y a pas d'activité malicieuse; ceci est bien différent du concept d'*assurance de sécurité*

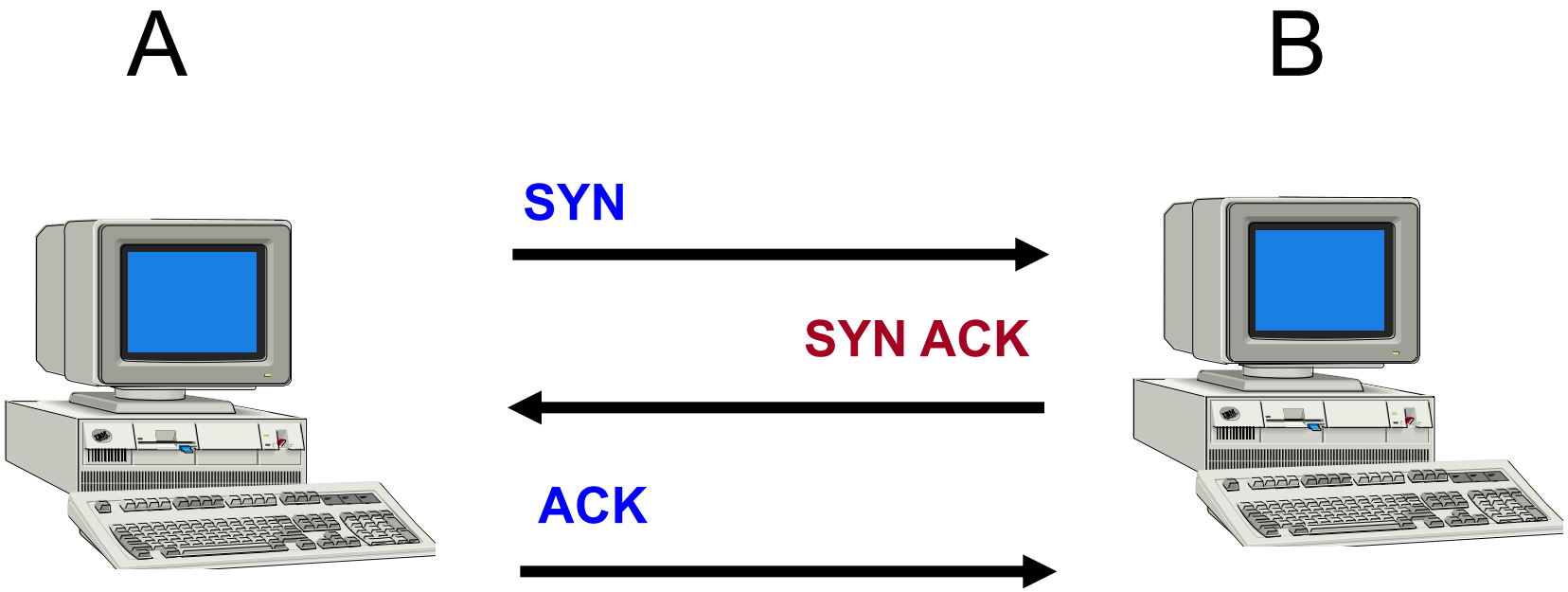
TCP Header Format

bits	0-3	4-9	10-15	16-31
0	Source Port			Destination Port
32	Sequence number			
64	Acknowledge number			
96	Data Offset	ECN	Control bits U|A|P|R|S|F	Window Size
128	Checksum			Urgent Pointer
160	Options (and padding)			
160 or 192	Data			

TCP Control Bits (*Flags*)

URG	**ACK**	**PSH**	**RST**	**SYN**	**FIN**
Urgent	Acknowledge	Push	Reset	Synchronize	Finish

Connexion de session TCP

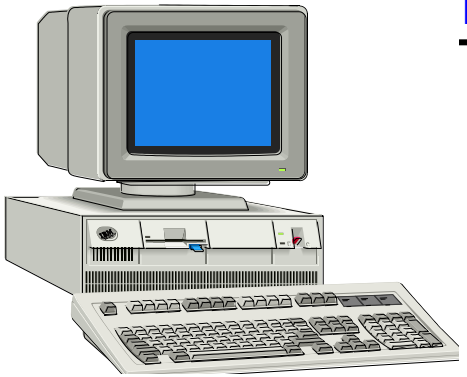


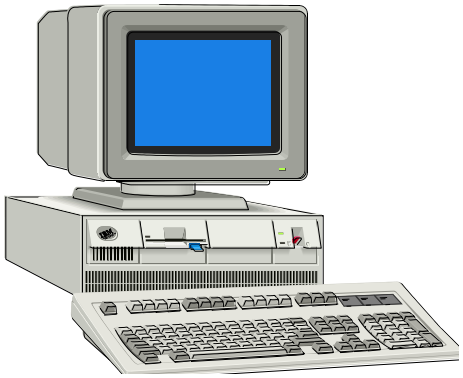
Connexion en trois étapes
The Three Way Handshake

No. Séquence et accusé réception (*ACK*)

A

B





SYN Seq = X →

← SYN ACK Seq = Y, Ack = X+1

ACK Seq = X + 1, Ack = Y+1 →

Data(1 octet) Seq = X+1, Ack = Y + 1 →

← Data(10 octets) Seq = Y+1, Ack = X+2

← Data(10 octets) Seq = Y+11, Ack = X+2

ACK Seq = X+2, Ack = Y+11 →

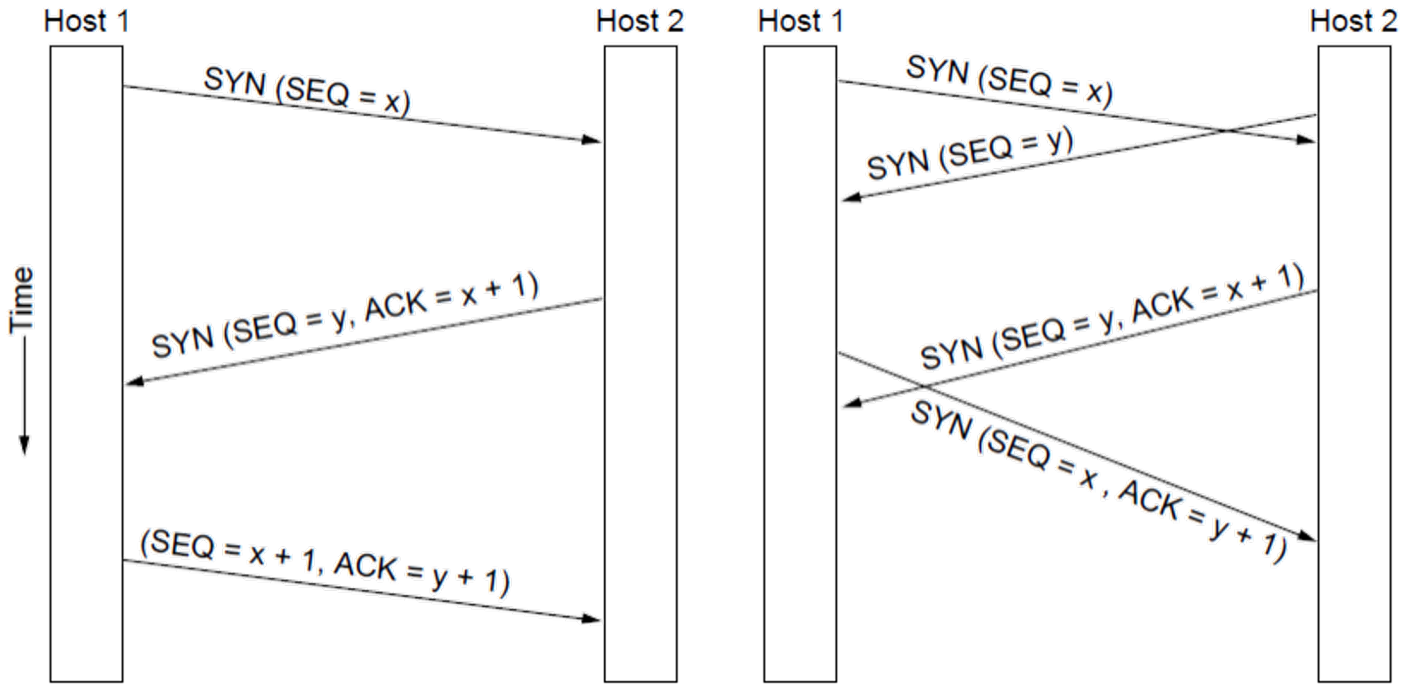
ACK Seq = X+2, Ack = Y+21 →

← Data(10 octets) Seq = Y+21, Ack = X+2

Les SYN et les FIN utilise un octet de données virtuel;
les ACK n'en transmettent pas
x et y sont des valeurs initiales aléatoires

Établissement de connexion TCP simultanée

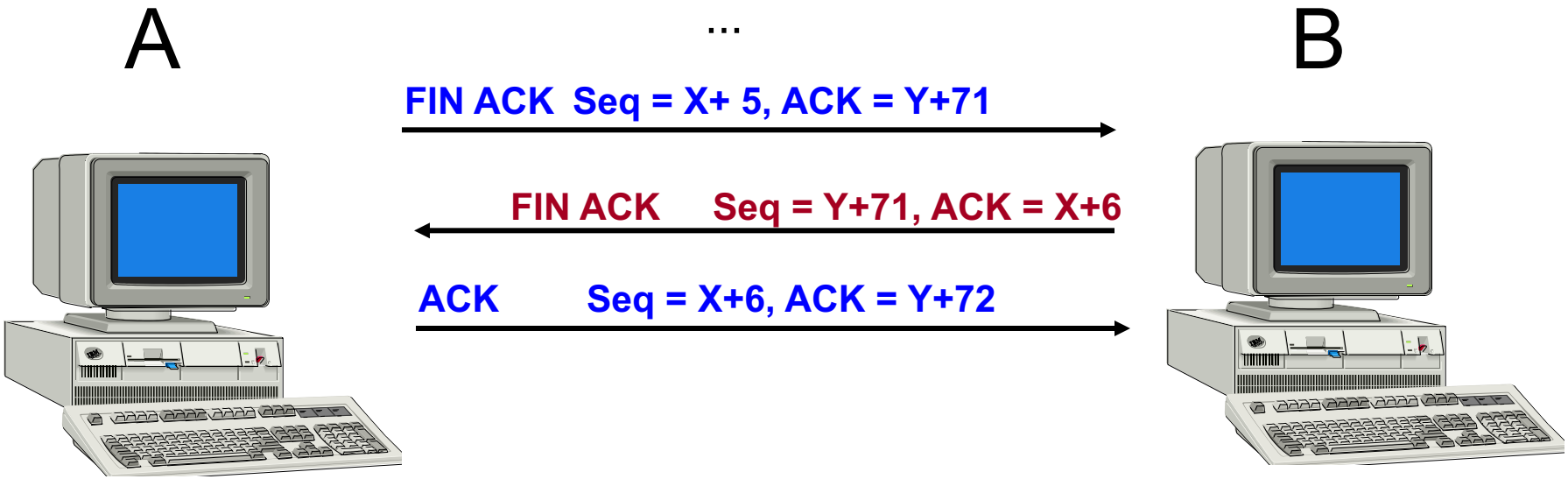
TCP peut également établir des connexions simultanées



Cas habituel

Connexions simultanées

Fermer une connexion



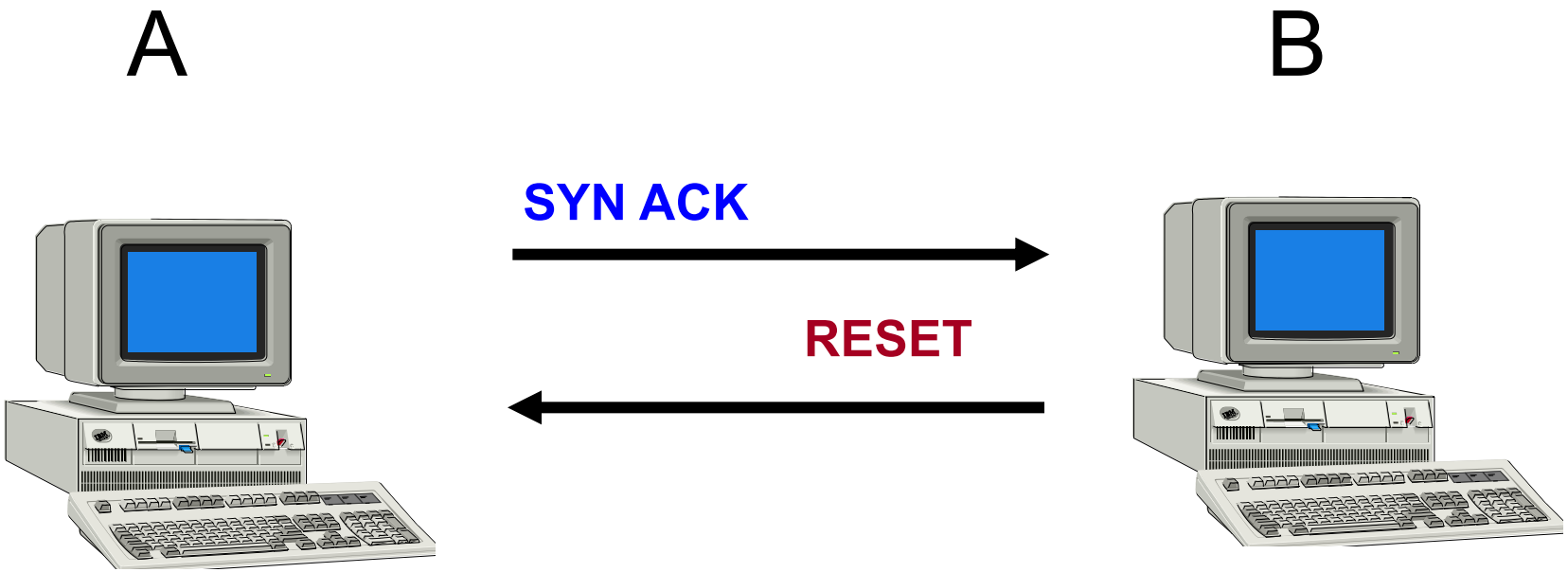
Noter que toutes données perdues peuvent être transmises après qu'une connexion soit fermée

Remise à zéro d'une connexion

- L'indicateur *reset*, RST, est utilisé pour remettre une connexion à zéro lorsqu'une erreur de connexion est détectée entre deux hôtes.

- Une remise à zéro peut être de mise lorsque:
  - Un segment TCP est reçu pour lequel le serveur n'a pas de connexion avec l'émetteur
    - sauf pour un SYN
  - Un segment TCP contient de l'information invalide
    - Tel numéro de séquence ou ACK invalides.
  - Un SYN TCP est reçu sur un port auquel aucun service n'est prêt à établir une connexion

TCP Reset Connection



Comment pourrait-on utiliser ceci malicieusement?

TCP PUSH Flag

- TCP accepte généralement les données de la couche application et la transmet comme flux sur la connexion, peu importe la structure des données ou ce qu'elles représentent.

- Ceci est parfois incompatible avec les exigences de l'application
  - Par exemple, une session `Telnet` ne fonctionnerait pas si les données de la ligne de commande étaient mis en mémoire tampon par TCP avant d'être envoyées au serveur.

- Lorsque la couche application exige qu'un bloc de données soit envoyé immédiatement, l'indicateur PUSH est utilisé pour forcer la transmission, et pour en avertir le récepteur
  - Dans le cas de `Telnet`, TCP pousse chaque caractère client sur la connexion
    - On voit donc ainsi l'écho de chaque caractère du nom utilisateur `Telnet`

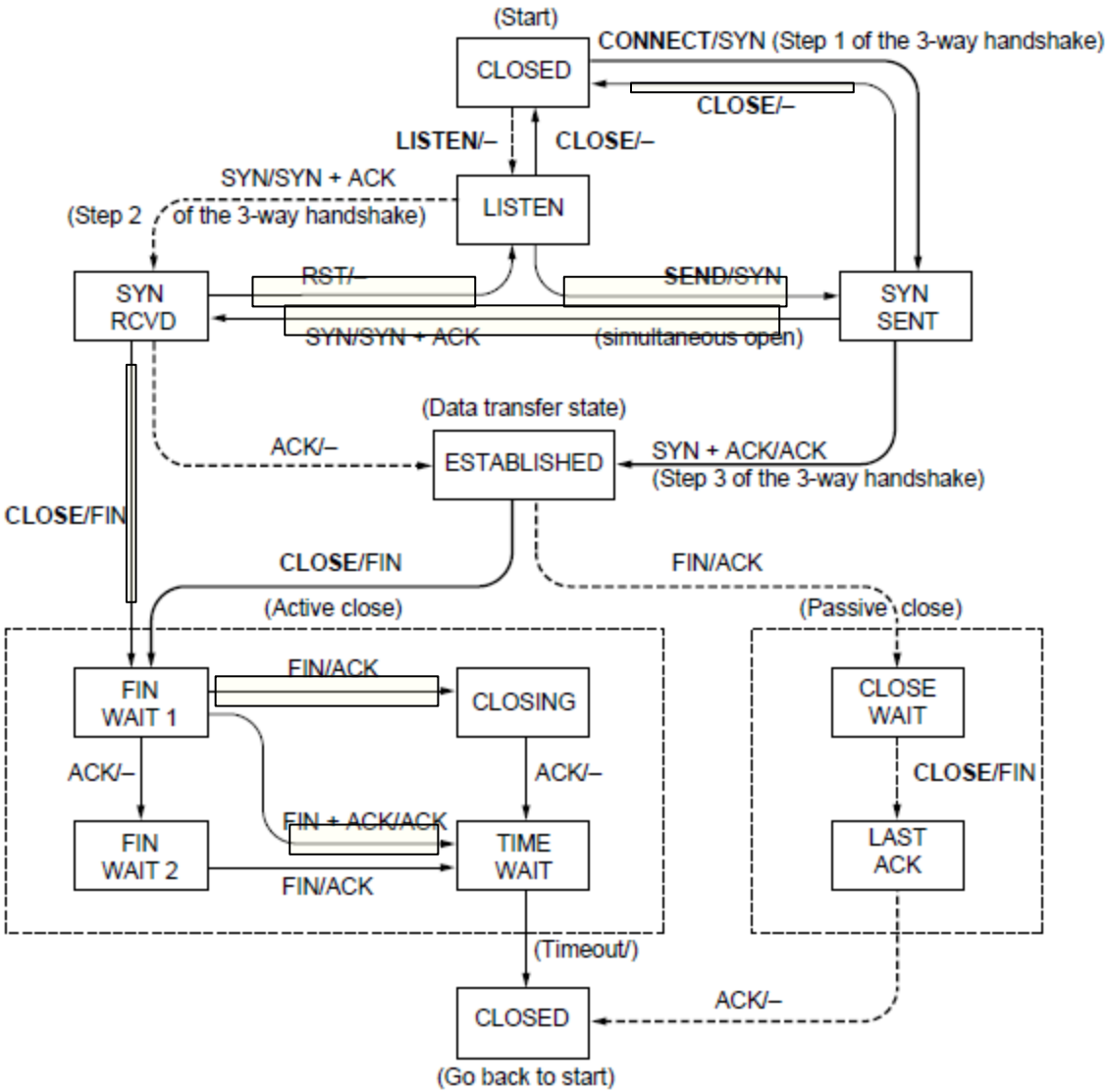
Modèle d'état de connexion TCP

Les lignes solides montrent le sentier habituel pour le client

Les lignes pointillées montrent le sentier habituel pour le serveur

Les lignes surlignées montrent des évènements inhabituels.

Les transitions sont étiquetées avec la cause/action.



Option TCP *Maximum Segment Size*

- Bien que la fenêtre dynamique TCP (WS) sera discutée lors de la prochaine séance, l'option TCP *Maximum Segment Size* (MSS) annonce le montant de données maximales qui peuvent être reçues dans un segment sans fragmentation sur le réseau local.
- Cette option est directement reliée au MTU du réseau local.
  - Par exemple, un hôte sur un réseau local typique Ethernet a un MTU de 1500. Considérant un en-tête IP de 20 octets et un en-tête TCP de 20 octets, le MSS serait de 1460 octets pour les données.

Un protocole axé sur HTTP:

QUIC

Internet vieillissant

- Rappel: TCP et UDP ont été conçus dans les années 80.
- La nature du trafic qui circule les réseaux informatiques et Internet a changé radicalement, même depuis l'an 2000.
- Considérez:
  - Netflix/YouTube/Twitch/Crave/Disney+… etc.
  - Diffusions d'événements VR
  - Steam Remote Play
  - Google Stadia
  - Consoles de machines virtuelles (comme notre labo…)
- Y a-t-il des problèmes d'utilisation de ces services avec TCP / UDP «basiques»?

QUIC/HTTP3

- Protocole de la couche transport, conçu à l'origine par Google.
- Les premières versions de QUIC sont utilisées par YouTube/Google depuis 2013.
- Actuellement en cours de travail / soutenu par plusieurs entreprises et en bonne voie pour devenir une norme IETF*.
- Comprend plus de la moitié de toutes les connexions entre Chrome et les services Google.

* Internet Engineering Task Force

QUIC/HTTP3

- Basé sur le protocole UDP, mais optimisé pour le trafic de type HTTP.
- Certains mécanismes sont déplacés vers la couche application:
  - Détection d'erreur,
  - Correction des erreurs,
  - Contrôle de la congestion.
- Permet aux applications un contrôle plus fin des flux de données.
  - Peut crypter des paquets individuels vs. flux entier.
  - Les applications individuelles décident de supprimer ou de récupérer les paquets perdus.
- Conçu pour plusieurs flux entre client/serveur. Si un flux doit corriger une erreur, les autres continuent.

Prochaine séance

ARCHITECTURE: DÉFENSE DE PÉRIMÈTRE